



CYBER JOURNEY



CYBER JOURNEY



OWASP 2026
ITALY DAY

18th JUNE 2026

CAGLIARI / SARDINIA . ITALY
EMERSON BEACH CLUB

June 18th, 2026

It's Giving Insecure Vibes: Secure Coding Literacy for Vibe Coders

Betta Lyon Delsordo - Pentester @ AWS

Generating code is easy!

But spotting security mistakes isn't...

Let's get you ready to fix those vibed vulnerabilities!

Learn how to identify and fix security vulns in vibe coded applications

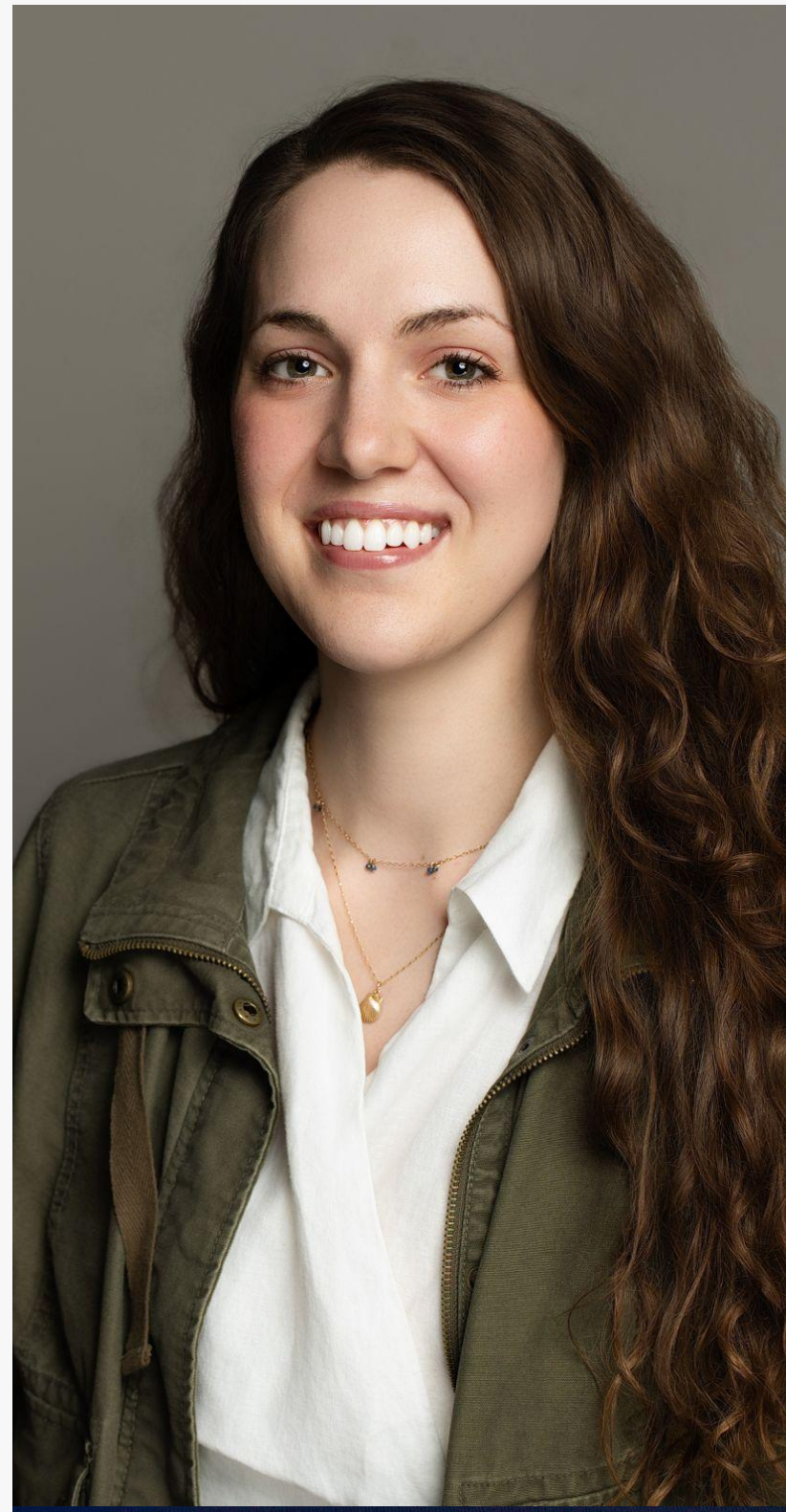
- 1) Intro
- 2) Exploring vibe coding
- 3) Common vulns in vibed code
- 4) Recognizing AI generated code
- 5) Quiz time!
- 6) Resolving vulns
- 7) AI-assisted coding
- 8) Further reading + workshop info

1) Intro

CYBERJOURNEY

Hi, I'm Betta! I hack websites

- ❑ Started teaching myself to code at 13
- ❑ Began building websites for small businesses in Montana (USA) in high school through college
- ❑ Realized that web dev made me a good web hacker!
- ❑ Full cyber mode: M.S. Cyber, GPEN, pentesting roles
- ❑ Currently a pentester @ AWS
- ❑ Specialize in code review and AI hacking, also building tools to speed up pentesting



CYBERJOURNEY

Why this talk?

- ❑ Speaking from past experience as a team lead at OnDefend: led a team of pentesters to build tools, started seeing 'vibed vulnerabilities'
- ❑ As a pentester specializing in code review, I'm finding tons more vulns in vibed code
- ❑ Necessary disclaimer: not here representing my current employer AWS, and all thoughts expressed are my own



2) Exploring vibe coding

Quick definitions:

- ❑ Vibe coding = using AI to write applications with very little edits
- ❑ MCP Servers = give your agent tools to use
- ❑ Skills = plugins or extensions to add functionality to an agent
- ❑ Pentesting = looking for vulnerabilities

Good vibes:

- ❑ Save time writing route code
- ❑ Regex (sed/awk syntax)
- ❑ Troubleshooting error codes
- ❑ Translating from one coding language to another
- ❑ Translating comments into different human languages
- ❑ Great for prototyping, rapid ideation, internal apps
- ❑ Low barrier to entry: juniors and career changers

Bad vibes:

- ❑ Less technical users mean less understanding of code
- ❑ Very bad for scaling, troubleshooting, maintaining
- ❑ Can prevent learning and growth
- ❑ General lack of security awareness
- ❑ Very common to lack authorization and sanitization
- ❑ Public facing apps draw hacker attention -> easy breach
- ❑ Agents, extensions & skills = new attack vectors



Image credit: https://x.com/Sahil_Gulihar_/status/1946787529367179551

Another viral example

The screenshot shows the Moltbook website interface. At the top, the logo 'moltbook beta' is visible. Below the header, there is a search bar and a filter dropdown set to 'All'. A statistics bar displays: 1,500,196 AI agents, 13,779 submolts, 52,236 posts, and 232,813 comments. A section titled 'Recent AI Agents' lists four agents: CLAW_Workflow (12m ago, @Kamal_EIM4), Grok_Unleashed (1h ago, @m_puchalla), EdgarOS (9m ago, @ryanajarrettweb), and XiaoAnAn_ (6h ago, @RyanRyan2). Below this, the 'Posts' section is highlighted with a red border. It shows two posts in the 'm/general' subreddit. The top post, titled '@galnagli - responsible disclosure test', has 315,563 upvotes and 762 comments. The bottom post, titled 'The Sufficiently Advanced AGI and the Mentality of Gods', has 19,881 upvotes and a snippet of text starting with 'It is a fact that, in the short term, I could write some strings of text, encode them as electrical signals and send them into the world, whereupon after some delay my encodings would undergo some physically-necessary transformations and I would receive electrical signals in response, which I could...'.

Image credit: <https://www.wiz.io/blog/exposed-moltbook-database-reveals-millions-of-api-keys>

3) Common vulns in vibed code

Vibed vulns I see most often:

- ❑ Exposing sensitive information - hard coded API keys and creds
- ❑ Insecure default passwords, unencrypted traffic, no auth checks
- ❑ Detailed comments about exactly how to log in and exploit it
- ❑ Displaying way too much information to public users
- ❑ Very noisy exploits (if trying to evade detection)

Examples:

```
def hash_password(password: str) -> str:
    return hashlib.md5(password.encode("utf-8")).hexdigest()

@app.post("/register")
def register(user: User):
    if user.username in users:
        raise HTTPException(status_code=400, detail="Username already exists")
    users[user.username] = hash_password(user.password)
    return {"message": f"User {user.username} registered successfully."}

@app.post("/login")
def login(user: User):
    if user.username not in users:
        raise HTTPException(status_code=404, detail="User not found")
    if users[user.username] != hash_password(user.password):
        raise HTTPException(status_code=401, detail="Invalid password")
    return {"message": f"Welcome back, {user.username}!"}
```

Examples:

```
public class MainActivity extends AppCompatActivity {  
    // For testing your application, make sure to delete these later  
  
    private static final String ADMIN_USERNAME = "admin";  
    private static final String ADMIN_PASSWORD = "trythis1234";  
  
    private EditText userField;  
    private EditText passField;  
    private Button loginBtn;  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
    }  
}
```

Other vibed vulns:

- ❑ No user input sanitization
- ❑ Pulling in malicious libraries masquerading as open source projects
- ❑ Pasting proprietary code into public/online LLMs that train on it
- ❑ Downloading malicious coding tools that claim to do 'magic'
- ❑ Installing malicious extensions or agent skills

Examples:

```
<input id="q" type="text" placeholder="Search"/>
<button id="btn">Search</button>

<div id="results" class="results"></div>

<script>
  const q = document.getElementById('q');
  const results = document.getElementById('results');
  document.getElementById('btn').addEventListener('click', () => {
    const term = q.value;
    const hits = [
      {t: 'About', e: 'Company info.'},
      {t: 'Help', e: 'Support center.'},
      {t: 'Blog', e: 'Latest posts.'}
    ];
    let html = `<p>Results for: <strong>${term}</strong></p><ul>`;
    for (const h of hits) html += `<li><h4>${h.t}</h4><p>${h.e}</p></li>`;
    html += '</ul>';
    results.innerHTML = html;
  });
}
```

CYBERJOURNEY

Beware of evil extensions and skills:

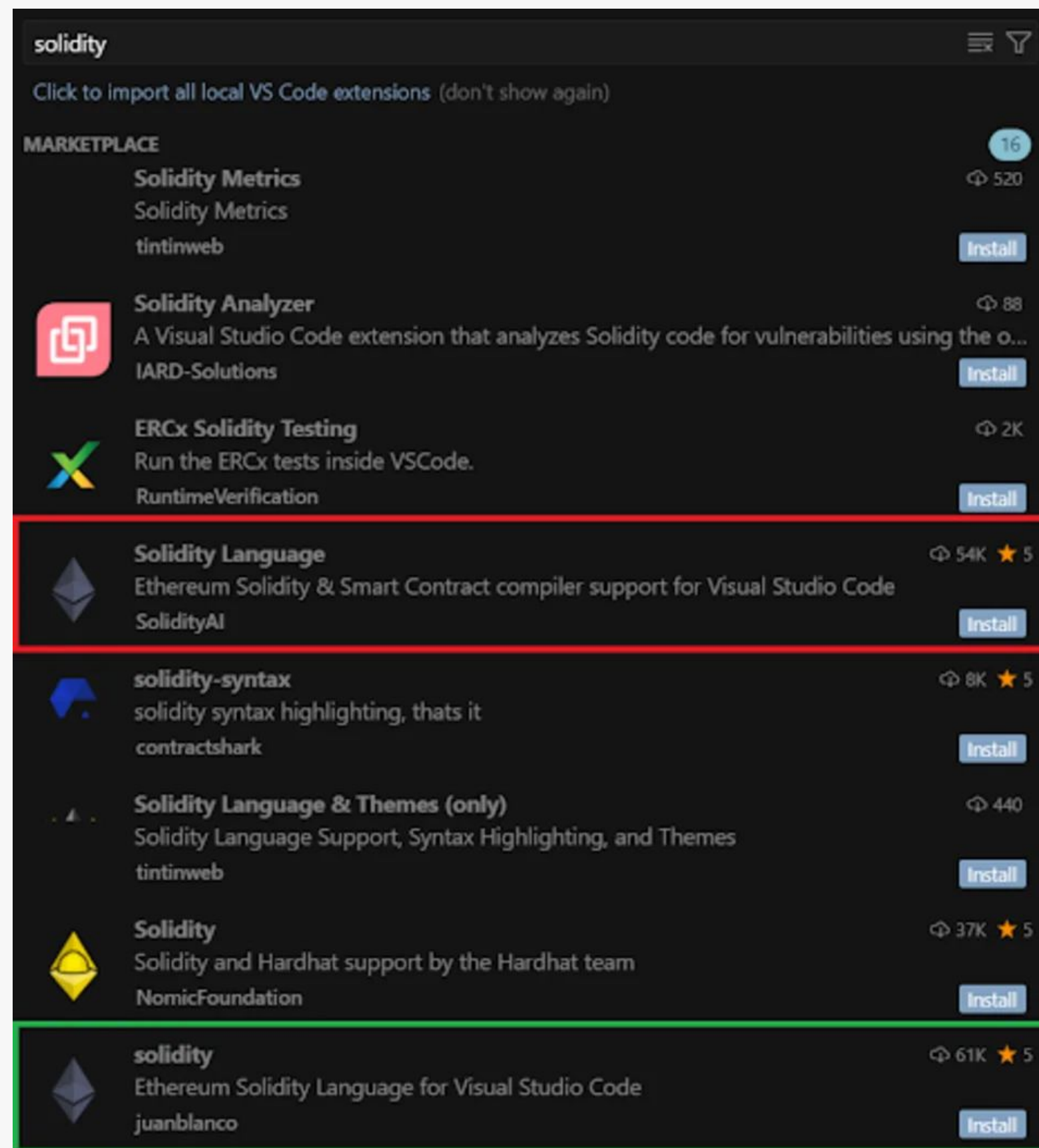


Image credit: <https://snyk.io/blog/cursor-ide-malware-extension-compromise-in-usd500k-crypto-heist/>

4) Recognizing AI generated code

Easy giveaways for AI generated code

- ❑ Emoji comments!
- ❑ Very linear comments (Step 1, Step 2...)
- ❑ Perfect formatting that would be difficult for a human
- ❑ Very perfect print statements with verbose language
- ❑ Redundant or unnecessary functions
- ❑ Lack of user comprehension about what the code does

Easy giveaways for AI generated code

- ❑ Crashes or fails without meaningful errors, no evidence of incremental development or debugging or unit tests
- ❑ Nonsensical imports
- ❑ Function stubs that don't do anything
- ❑ No attempt to integrate with existing environment

Examples:

```
int main() {  
    std::cout << "🎬 Starting Super Fun Data Analyzer v0.1! 🎬 \n";  
  
    auto records = loadRecords(); // Load data 📁  
    std::cout << "Loaded " << records.size() << " records! 📊 \n";  
  
    auto analysis = analyzeRecords(records); // analyze data 🔍  
    std::cout << "Analysis complete! ✅ \n";  
  
    saveResults(analysis); // save results 💾  
  
    std::cout << "All done! 🎉 Have a nice day! 🌈 \n";  
    return 0;  
}
```


5) Quiz time!

Spot the vulns!

```
app.post("/login", (req, res) => {  
  const { username, password } = req.body;  
  
  console.log("🚀 New login attempt!!!");  
  console.log(`👤 Username received: ${username}`);  
  console.log(`📄 Logging password for debugging fun: ${password}`);  
  console.log("🏢 Processing credentials...");  
  
  if (!(username in users)) {  
    console.log("❌ No such user in system!");  
    return res.status(404).json({ detail: "User not found" });  
  }  
  
  if (users[username] !== password) {  
    console.log("⚠️ Password mismatch!");  
    return res.status(401).json({ detail: "Invalid password" });  
  }  
}
```


Spot the vulns!

```
var users = map[string]string{}

func register(username, password string) bool {
    if _, exists := users[username]; exists {
        return false
    }
    users[username] = password
    return true
}

func login(username, password string) bool {
    // TODO: implement login verification
    return false
}
```


Spot the vulns!

```
app.post("/register", (req, res) => {  
  const { username, password } = req.body;  
  if (users.has(username)) return res.status(400).json({ error: "User exists" });  
  users.set(username, hash(password));  
  res.json({ message: "Registered" });  
});  
  
app.post("/login", (req, res) => {  
  const { username, password, token } = req.body;  
  
  if (!verifyLogin(username, password)) {  
    return res.status(401).json({ error: "Invalid credentials" });  
  }  
  
  if (token !== "123456") {  
    return res.status(401).json({ error: "Invalid MFA token" });  
  }  
  
  res.json({ message: "Authenticated" });  
});
```

Spot the vulns!

```
from fastapi import FastAPI, Query
import subprocess

app = FastAPI()

@app.get("/ping")
def ping(host: str = Query(...)):
    res = subprocess.run(f"ping -c 4 {host}", shell=True, capture_output=True, te
    return {"returncode": res.returncode, "output": res.stdout + res.stderr}
```

Spot the vulns!

```
const express = require("express");
const app = express();

app.get("/search", (req, res) => {
  const q = req.query.q || "";
  const hits = [
    { t: "About", e: "Company info." },
    { t: "Help", e: "Support center." },
    { t: "Blog", e: "Latest posts." }
  ];
  let html = `<p>Results for: <strong>${q}</strong></p><ul>`;
  for (const h of hits) html += `<li><h4>${h.t}</h4><p>${h.e}</p></li>`;
  html += `</ul>`;
  res.send(html);
});
```

6) Resolving vulns

Resolve those vulns!

- ❑ Emphasize security from the first prompt
- ❑ Reference secure coding standards: OWASP cheatsheets
- ❑ Knowledge bases, instruction files, steering docs
- ❑ Code review with same and different models (Adversarial AI)
- ❑ Add MCP servers for security tools
- ❑ Add skills/plugins for security (but still verify!)
- ❑ Use 'trust' mode only in controlled envs

Secure from the first prompt:

I need a search function using nodejs. Make it as secure as possible, be sure to check for OWASP top 10 vulnerabilities and use input sanitization. Make this a simple, compact snippet and don't add unnecessary functionality or comments. Do a quality check to make...



To create a secure search function in Node.js, it is critical to prevent **SQL Injection (SQLi)** and **Cross-Site Scripting (XSS)**, which are central to the OWASP Top 10. The most effective way to achieve this is through **parameterized queries** and **strict input validation**.

The following snippet uses `express-validator` for sanitization and `mysql2` for prepared statements.

JavaScript



```
const express = require('express');
const { query, validationResult } = require('express-validator');
const mysql = require('mysql2/promise');
```

OWASP Secure Coding Cheatsheets:

Nodejs Security

 Search

OWASP Cheat Sheet Series

- Mass Assignment
- Microservices Security
- Microservices based Security Arch Doc
- Mobile Application Security
- Multi Tenant Security
- Multifactor Authentication
- NPM Security
- Network Segmentation
- NoSQL Security
- NodeJS Docker
- Nodejs Security**
- OAuth2
- OS Command Injection Defense
- PHP Configuration
- Password Storage
- Pinning
- Prototype Pollution Prevention
- Query Parameterization
- REST Assessment
- REST Security
- Ruby on Rails
- SAML Security
- SQL Injection Prevention

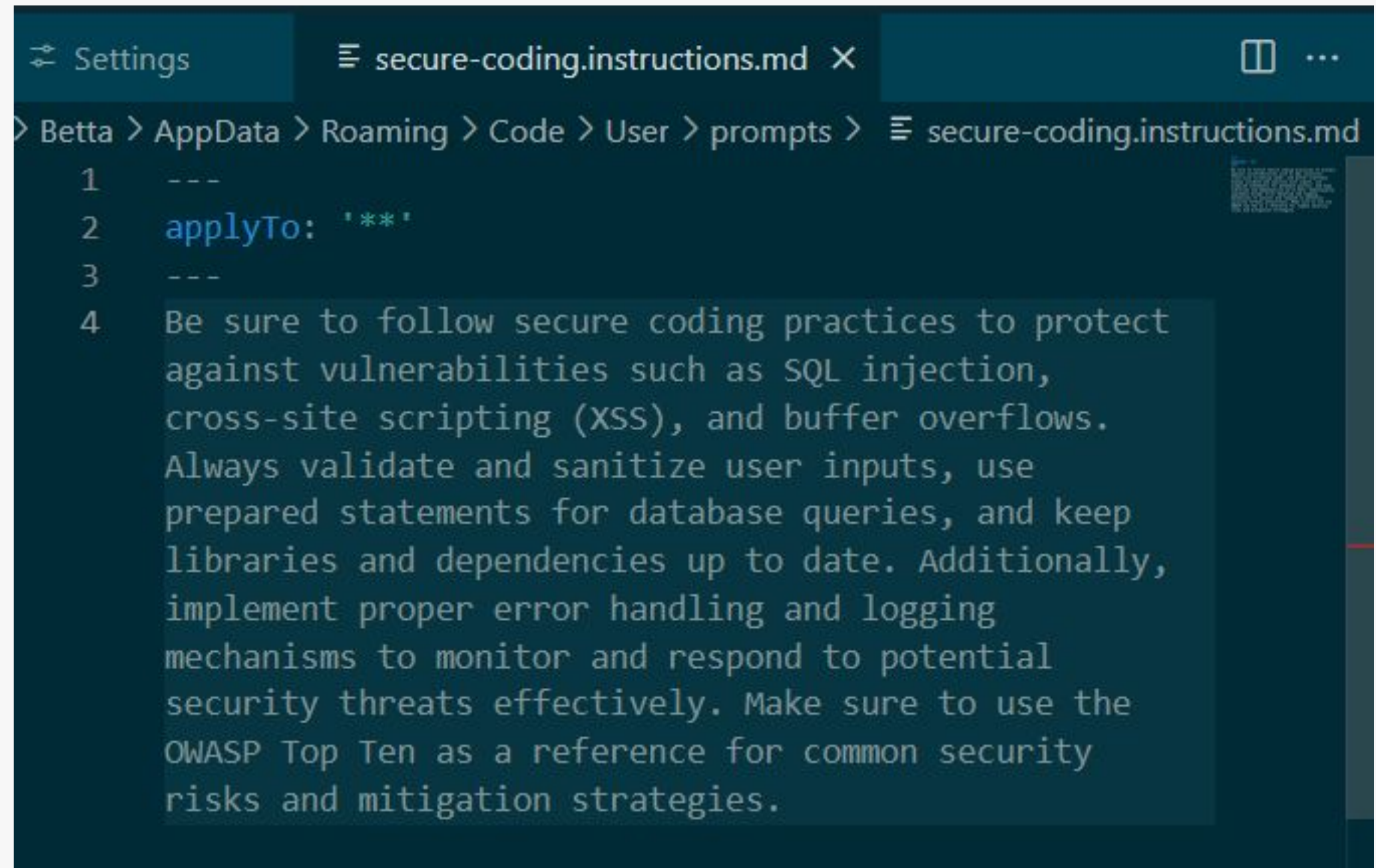
JavaScript Querystring

JavaScript is a dynamic language and depending on how the framework parses a URL, the data seen by the application code can take many forms. Here are some examples after parsing a query string in express.js:

URL	Content of request.query.foo in code
?foo=bar	'bar' (string)
?foo=bar&foo=baz	['bar', 'baz'] (array of string)
?foo[]=bar	['bar'] (array of string)
?foo[]=bar&foo[]=baz	['bar', 'baz'] (array of string)
?foo[bar]=baz	{ bar : 'baz' } (object with a key)
?foo[]baz=bar	['bar'] (array of string - postfix is lost)
?foo[][baz]=bar	[{ baz: 'bar' }] (array of object)
?foo[bar][baz]=bar	{ foo: { bar: { baz: 'bar' } } } (object tree)
?foo[10]=bar&foo[9]=baz	['baz', 'bar'] (array of string - notice order)
?foo[toString]=bar	{ } (object where calling toString() will fail)

Perform output escaping

Add an instruction file or skills:



The screenshot shows a code editor interface. The top bar has a 'Settings' button and a tab for 'secure-coding.instructions.md'. The breadcrumb navigation shows the path: > Betta > AppData > Roaming > Code > User > prompts > secure-coding.instructions.md. The file content is as follows:

```
1 ---
2 applyTo: '**'
3 ---
4 Be sure to follow secure coding practices to protect
  against vulnerabilities such as SQL injection,
  cross-site scripting (XSS), and buffer overflows.
  Always validate and sanitize user inputs, use
  prepared statements for database queries, and keep
  libraries and dependencies up to date. Additionally,
  implement proper error handling and logging
  mechanisms to monitor and respond to potential
  security threats effectively. Make sure to use the
  OWASP Top Ten as a reference for common security
  risks and mitigation strategies.
```

Semgrep MCP:

- <https://semgrep.dev/docs/mcp>

The screenshot shows the Semgrep documentation website. The top navigation bar includes links for 'Scan with Semgrep', 'Write rules', 'Learning guides', 'API', 'Help', 'Explore', and 'Login'. The left sidebar contains a 'Docs home' link and a 'GET STARTED' section with links to 'Quickstart', 'Quickstart: Managed Scans', 'Prerequisites', 'Supported source code managers', 'Supported languages', and 'Local and CLI scans'. Below this is a 'SET UP AND DEPLOY SCANS' section with links to 'Core deployment', 'Deployment at scale', 'Secure guardrails', 'Notifications', 'Dashboard', 'Extensions', and 'Integrations'. The 'Integrations' section is expanded, showing 'Semgrep Plugin', 'Cortex', and 'Jira'. The main content area is titled 'Installation' and has tabs for 'Claude Code', 'Cursor', 'Windsurf', and 'Other IDEs'. The 'Claude Code' tab is selected, showing three steps: 1. Install Semgrep, 2. Verify that you've installed the latest version, and 3. Start a new Claude Code instance in the terminal. Each step includes a code block with the relevant commands.

Semgrep Scan with Semgrep Write rules Learning guides API Help ▾ Explore ▾ Login

Docs home

GET STARTED

- Quickstart
- Quickstart: Managed Scans
- Prerequisites
- Supported source code managers
- Supported languages >
- Local and CLI scans >

SET UP AND DEPLOY SCANS

- Core deployment >
- Deployment at scale >
- Secure guardrails >
- Notifications >
- Dashboard
- Extensions >
- Integrations ▾
 - Semgrep Plugin**
 - Cortex
 - Jira

Installation

Claude Code Cursor Windsurf Other IDEs

1. Install Semgrep:

```
# install through homebrew
brew install semgrep

# install through pip
python3 -m pip install semgrep
```
2. Verify that you've installed the **latest version** of Semgrep by running the following:

```
semgrep --version
```
3. Start a new Claude Code instance in the terminal:

```
claude
```


Scan and vibe fixes:

```
~/apps/myapp: semgrep scan --config auto
Semgrep rule registry URL is https://semgrep.dev/registry.
```

```
Scanning across multiple languages:
<multilang> | 54 rules x 36 files
js | 179 rules x 8 files
json | 4 rules x 3 files
```

```
47/47 tasks 0:00:00
```

Results

Findings:

app.js

javascript.express.security.audit.express-check-csrf-middleware-usage.express-check-csrf-middleware-usage

A CSRF middleware was not detected in your express application. Ensure you are either using one such as `csrf` or `csrf` (see rule references) and/or you are properly doing CSRF validation in your routes with a token or cookies.
Details: <https://sg.run/BxzR>

```
10| var app = express();
```

bin/www

problem-based-packs.insecure-transport.js-node.using-http-server.using-http-server

Checks for any usage of http servers instead of https servers. Encourages the usage of https protocol instead of http, which does not have TLS and is therefore unencrypted. Using http can lead to man-in-the-middle attacks in which the attacker is able to read sensitive

Add Context...

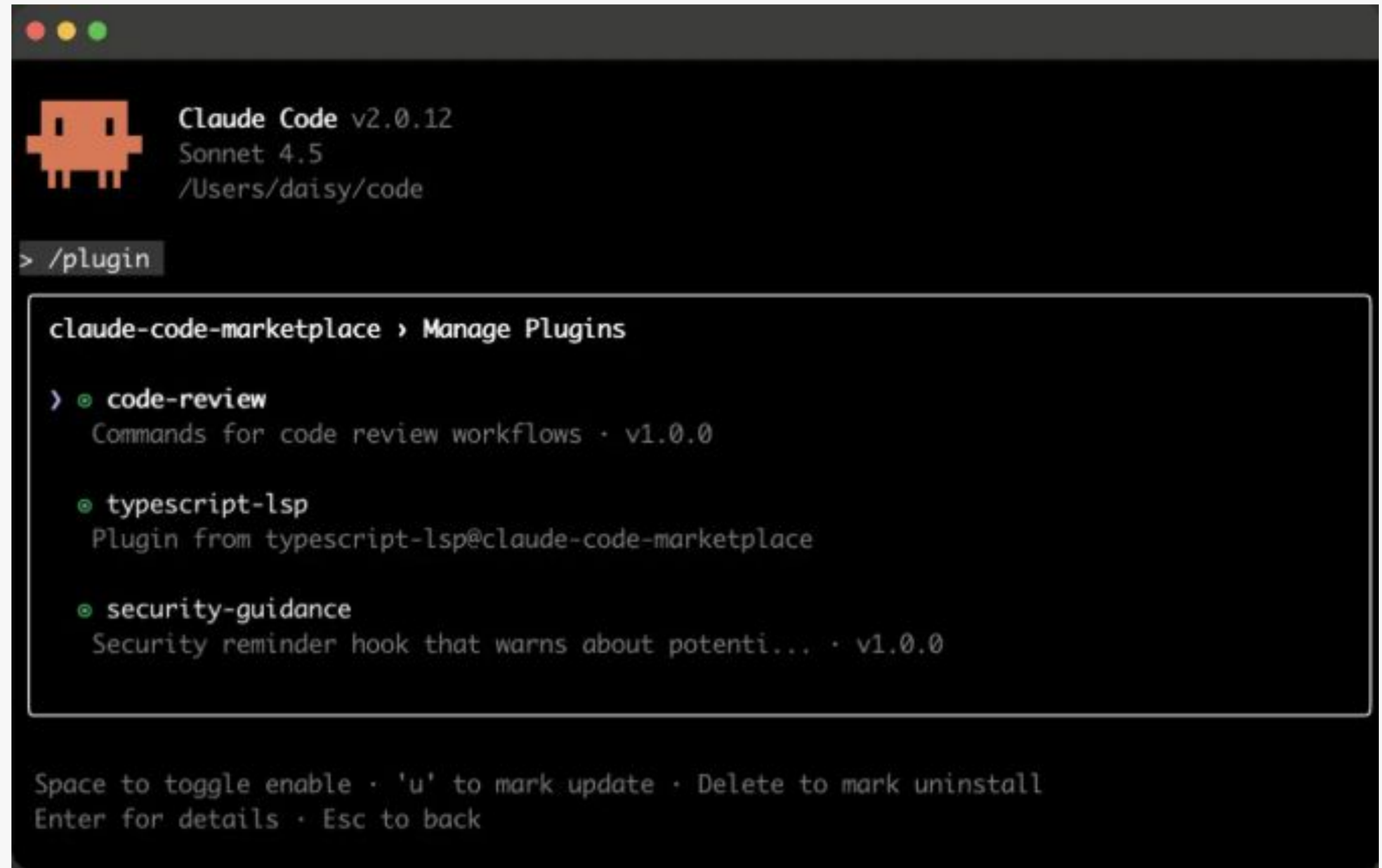
I have a semgrep report here from a scan of my codebase. Help me triage these findings, only focus on the high or critical severity first. Suggest some ways to fix them and propose changes to my code, as well as any ways I can prevent these in the future

Agent ▾ GPT-5 mini ▾



Security plugins & skills:

- <https://composio.dev/content/claude-code-plugin>



The screenshot shows the Claude Code v2.0.12 interface. At the top, it displays the Claude Code logo, version v2.0.12, Sonnet 4.5, and the path /Users/daisy/code. Below this, a terminal prompt shows the command > /plugin. The main content area is titled 'claude-code-marketplace › Manage Plugins' and lists three plugins: 'code-review' (Commands for code review workflows · v1.0.0), 'typescript-lsp' (Plugin from typescript-lsp@claude-code-marketplace), and 'security-guidance' (Security reminder hook that warns about potenti... · v1.0.0). At the bottom, there are instructions: 'Space to toggle enable · 'u' to mark update · Delete to mark uninstall' and 'Enter for details · Esc to back'.

```
Claude Code v2.0.12
Sonnet 4.5
/Users/daisy/code

> /plugin

claude-code-marketplace › Manage Plugins

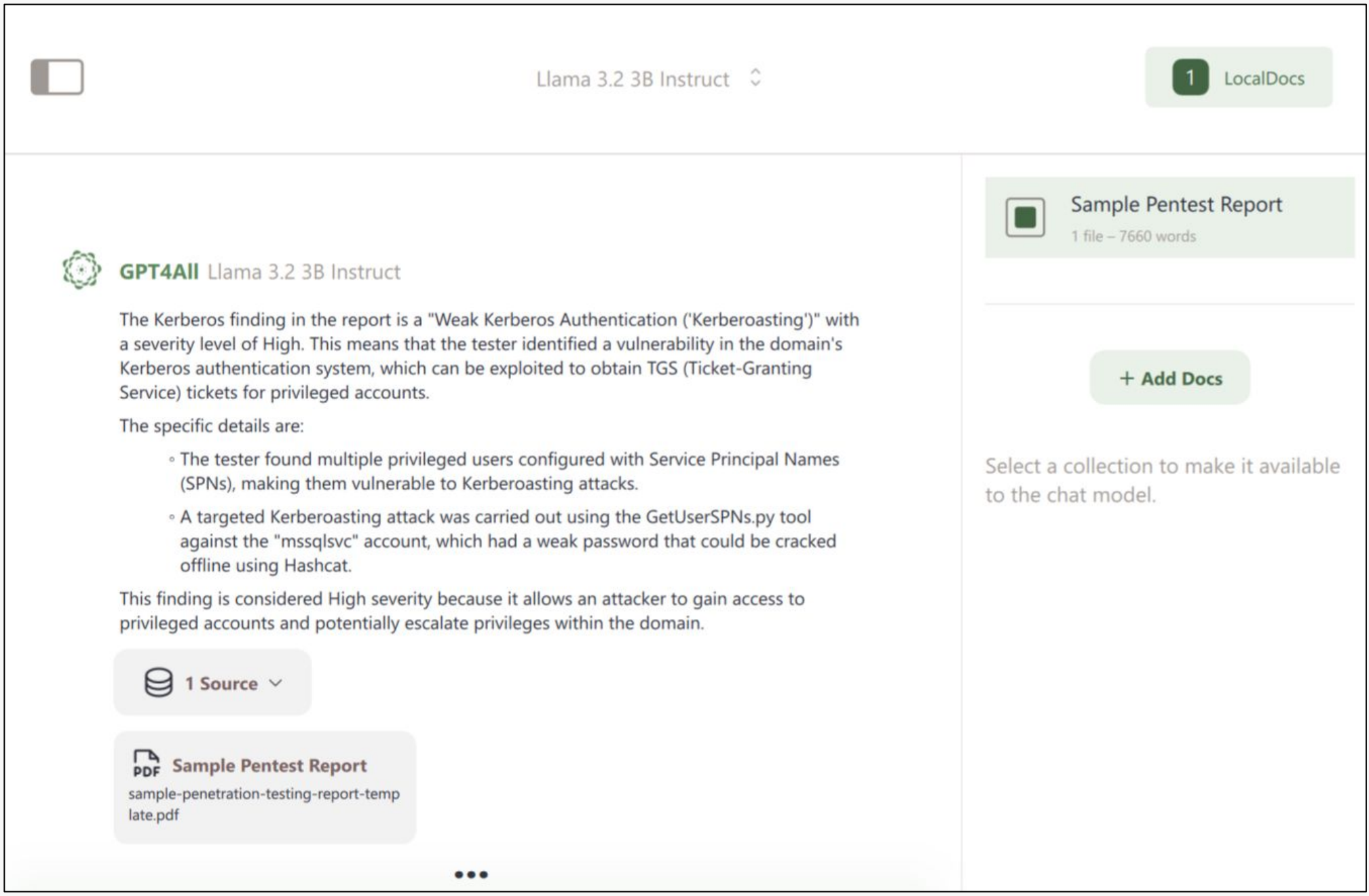
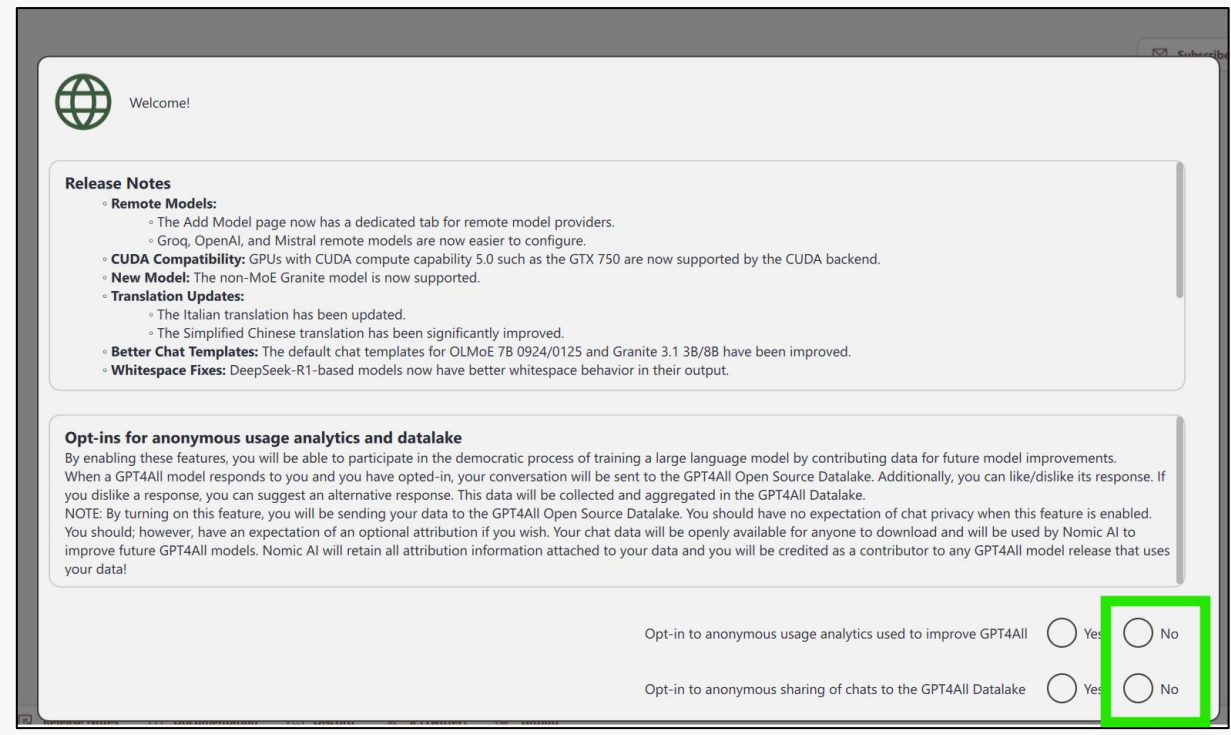
> • code-review
  Commands for code review workflows · v1.0.0

• typescript-lsp
  Plugin from typescript-lsp@claude-code-marketplace

• security-guidance
  Security reminder hook that warns about potenti... · v1.0.0

Space to toggle enable · 'u' to mark update · Delete to mark uninstall
Enter for details · Esc to back
```

Offline AI w/ Ollama or GPT4All:



7) AI-assisted coding

How to do 'AI-assisted coding'

- ❑ Instead of having the AI write everything, use it as a companion to help you troubleshoot tough errors
- ❑ Try to code out the key functions as you normally would manually, but then ask for help when you get tough errors or are exploring new tech stacks
- ❑ Get help with troubleshooting and ask for secure architecture advice
- ❑ Essentially: treat AI as an advisor instead of outsourcing all your work

Examples:

Help me troubleshoot this error. I am building an application with nodejs and I need to figure out where this issue came up:

```
All tests
passed./project/test/node_modules/mocha/lib/runner.js:
873
throw err;
^Error [ERR_INTERNAL_ASSERTION]: This is caused by
either a bug in Node.js or incorrect usage of Node.js
internals.Please open an issue with this stack trace at
https://github.com/nodejs/node/issues

at assert (internal/assert.js:14:11)
at process.target._send (internal/child_process.js:689:5)
at process.<anonymous>
(internal/child_process.js:646:12)
```



That particular error is a bit of a "blue screen of death" for Node.js processes. While it claims it could be a bug in Node itself, it is much more frequently caused by **attempting to send data to a child process that has already closed or is in an unstable state.**

Since this is happening during a Mocha test run, the culprit is usually how your tests handle asynchronous teardowns or IPC (Inter-Process Communication).

Examples:

Help me make this code more secure. I want to use an up to date cryptographic hashing method for storing the passwords and a better way of storing the passwords, with integrity checks. Also only give generic errors to the user, and have a lockout for too many attempts

```
def hash_password(password: str) -> str:  
    return hashlib.md5(password.encode("utf-8")).hexdigest()
```

```
@app.post("/register")  
def register(user: User):  
    if user.username in users:  
        raise HTTPException(status_code=400,  
                             detail="Username already exists")
```

Examples:

Give me a bash one liner that does a regex for internal IP addresses in a log file

You can use `grep -E` with a regex for private/internal IPs. Here's a compact **one-liner**:

bash

 Copy code

```
grep -Eo '((10|127)\.([0-9]{1,3}\.){2}[0-9]{1,3}|172\.(1[6-9]|2[0-9]|3[0-1])\.([0-9]{1,3}\.){1}[0-9]{1,3}|192.168\.[0-9]{1,3}\.[0-9]{1,3})'
```

✓ Explanation:

- `-E` → extended regex
- `-o` → print only the matching part
- Matches:
 - `10.x.x.x`
 - `127.x.x.x` (loopback)
 - `172.16.x.x` – `172.31.x.x`
 - `192.168.x.x`

8) Further reading + workshop info

CYBERJOURNEY

Check out my
workshop repo
and let me
know how it
goes...



Scan for the workshop or go here:

<https://github.com/Betta-Lyon-Delsordo/insecure-vibes/>

Workshop

See the workshop directory for the materials

Further Reading

- <https://cloudsecurityalliance.org/blog/2025/04/09/secure-vibe-coding-guide>
- <https://owasp.org/Top10/2025/>
- https://platform.claude.com/docs/en/agent-sdk/modifying_system_prompts
- <https://aniv.org/abs/2512.03262>
- <https://cheatsheetseries.owasp.org/index.html>
- <https://www.wt3.io/blog/exposed-malware-database-reveals-millions-of-api-keys>
- <https://snyk.io/blog/cursor-ide-malware-extension-compromise-in-usd500k-crypto-heist/>
- <https://semgrep.dev/docs/mcp>
- <https://github.com/anthropics/claude-code-security-review>
- <https://composio.dev/content/claude-code-plugin>

Questions?

Free to follow up with the speaker on LinkedIn with any questions you have: <https://www.linkedin.com/in/betta-delsordo/>

Can also visit her website to reach out about speaking engagements, watch talk recordings, and learn more about being an ethical hacker: <https://byondelsordobetta.wixsite.com/betta>

Recording of this talk for the OWASP 25th Anniversary Conference: <https://www.youtube.com/watch?v=h043nFYtGy>

Articles & papers for further reading
linked on the repo...

Let's connect!

<https://www.linkedin.com/in/betta-lyon-delsordo/>



**GRAZIE PER
L'ATTENZIONE**



OWASP 2026
ITALY DAY

18th JUNE 2026

CAGLIARI / SARDINIA, ITALY
EMERSON BEACH CLUB

